# Learning to Schedule: A Value-Aware Policy Discovery Experiment with Provable Queue-Global Necessity

Vedant Upadhyay
M.S. Computer Science, University of California Santa Cruz
Graduate Capstone Project, 2024–2025
veupadhy@ucsc.edu

Scott Brandt
Department of Computer Science and Engineering
University of California Santa Cruz
sbrandt@ucsc.edu

March 2026

## Abstract

CPU schedulers face a basic tension: you can optimize for throughput or for fairness, but classical approaches struggle to do both at once. Existing metrics like Jain's Fairness Index (JFI) and slowdown variance (SDV) make this worse—they miss important types of unfairness entirely.

We train a two-head attention Deep Q-Network (Value-Aware DQN, VA-DQN) on the Alibaba 2018 cluster trace. Each task gets an urgency curve $V_i(d) = \text{base}_i \cdot \max(\text{floor}_i, e^{-d/\tau_i})$, and we reward the agent for how much value it preserves across the whole queue at each step.

We prove the Quantum Scoring Monotonicity Lemma: any scoring function that looks at one task in isolation will always pick either the smallest or the largest quantum—never something in between. Three ablation experiments confirm this. Breaking out of this trap requires looking at the whole queue, which is exactly what our attention mechanism does.

VA-DQN hits $21.71 \pm 0.64$ s mean completion time across 3 independent training seeds (CV = 2.9%), beating Round Robin with zero task starvation. Our best single checkpoint reaches 17.23 s (N = 1500 evaluation episodes).

We also introduce a new fairness metric, the Value-Rate Fairness Index (VRFI). Here is the key finding: ten tasks with identical wait times score JFI = 1.0 and SDV = 0—both metrics say "perfectly fair"—yet VRFI = 0.641 catches the real problem: tasks with urgent value curves lose value twice as fast as slow ones.

A preference-conditioned variant ($\omega$-DQN) beats the Multi-Level Feedback Queue scheduler (MLFQ) on both throughput (20.60 s vs. 21.59 s) and starvation (32.5% vs. 36.0%) at preference setting $\omega_s = 0.7$. No oracle information needed. No retraining to switch modes.

## 1 Introduction

A CPU scheduler makes one decision, over and over: which waiting process gets the CPU next, and for how long? The Linux Completely Fair Scheduler (CFS) tracks how much CPU time each process has received (via virtual runtime) and always schedules the one with the least. This approximates fairness over time. However, it does not account for job length, so it cannot optimize for metrics like average completion time the way shortest-job-first scheduling can. Short jobs still get CPU quickly due to frequent preemption, but CFS does not preferentially complete them early the way SRPT would.

SRPT (Shortest Remaining Processing Time) solves the throughput problem: always run the job closest to finishing. It minimizes mean completion time in theory. The catch is that SRPT needs to know how long each job will run, and in practice, schedulers never have that information. It also starves long jobs.

We ask a different question. Can an agent learn a scheduling policy from scratch—without copying an existing scheduler, without expert demonstrations, without hand-tuned priority rules—just from feedback about how long tasks have been waiting? We start by giving the agent oracle burst time (an upper bound) and then strip it away to see what the agent can do with only observable information.

This two-phase approach matters. It separates the question of what the reward signal can teach from the question of what information is actually available in

production. If the agent degrades gracefully when we remove oracle information, that tells us the reward design is doing real work.

### Agent Names

We use descriptive names throughout. **VA-DQN** (formerly W10C): 2-head attention DQN with oracle burst time. **VA-DQN-OBS** (formerly W12): same architecture, no burst time. **$\omega$-DQN** (formerly W14-$\omega$): preference-conditioned with fixed affine modulation. **$\omega$-DQN-VarN** (formerly W15): trained on variable-length Poisson arrival streams.

### Contributions

1. We document nine failure modes across four architectures—from tabular Q-learning to 2-head attention DQN—and diagnose each to a specific cause.

2. VA-DQN achieves $21.71 \pm 0.64$ s mean completion time (3 seeds, CV = 2.9%), with zero starvation. Best checkpoint: 17.23 s at N = 1500 episodes. Two attention heads spontaneously specialize—one surveys the queue, one tracks the longest-waiting task—without any supervision.

3. We prove the Quantum Scoring Monotonicity Lemma and confirm it with three ablations: any value-drop scorer collapses to always picking $q_{\min}$ or always $q_{\max}$.

4. We find that default $\tau$ values (25 s, 100 s) are off by a factor of 44× and 8× from actual trace delays. Even with calibrated values, 93–100% of ordering decisions are near-ties—a consequence of trace composition, not bad tuning.

5. VRFI (Value-Rate Fairness Index) catches unfairness that JFI and SDV both miss. We show a concrete case: JFI = 1.0, SDV = 0, yet VRFI = 0.641.

6. A 2×2 ablation shows that reward structure matters more than state observability. Swapping out the value-delta reward collapses SRPT agreement to 11.3%. Removing oracle burst costs roughly 4.4 s relative to the best checkpoint.

7. $\omega$-DQN improves both throughput and starvation relative to MLFQ at $\omega_s = 0.7$. One model, adjustable at runtime, no retraining.

8. $\omega$-DQN-VarN confirms the MCT advantage holds on variable-length queues (mean −1.40 s vs. MLFQ, 3 seeds).

# 2 Background

## 2.1 Why CFS Falls Short

CFS schedules the process with the smallest virtual runtime, approximating proportional CPU fairness across tasks. Fairness does not optimize for latency though: unlike shortest-job-first, CFS does not prioritize short tasks for early completion. Short jobs still get CPU quickly due to frequent preemption, but CFS does not explicitly favor them.

Since Linux 6.6, the default scheduler uses an EEVDF-based model [13], which adds virtual deadlines and eligibility times to more precisely control when tasks become runnable under fair scheduling. Our approach differs from both: instead of virtual runtime or deadlines, we model per-task urgency curves directly. That shifts the goal from time-fairness to value-fairness—tasks that lose value faster get scheduled sooner.

## 2.2 What Prior RL Schedulers Do

KernelOracle [5] trains an LSTM to predict which task CFS will select next—it learns to mimic an existing policy, not discover a new one. ALPS [6] approximates SRPT by simulating optimal schedules on historical workloads and enforcing the resulting priorities via eBPF, reporting a 57.2% latency reduction in serverless environments. RL-based schedulers like RLScheduler [7], Double DQN approaches [8], and SmartOS [9] learn policies using workload statistics, job classes, or multi-resource signals.

Most prior approaches rely on historical runtime information, workload-specific priors, or reference policies. Our approach uses only simple, online-computable features—remaining burst, wait time, and arrival indicators—without offline profiling or job-class metadata. One caveat: remaining burst time is oracle information in our experiments, unavailable in real kernels without estimation. Section 5.5 measures the exact cost of removing it: roughly 4.4 s MCT on the Alibaba 2018 trace.

## 2.3 Attention for Queue-Global Decisions

Standard DQN scores each action independently. That breaks down for scheduling because the right choice for task $i$ depends on what every other task in the queue needs. We use scaled dot-product attention [3] so each task sees the full queue before getting scored. Section 3 explains formally why this matters.

### 2.4 Production Systems Hit the Same Wall

A 2026 public preview of cloud batch scheduler fairness added a server-side backlog tracker because per-task metadata alone was not enough. That is exactly what our Corollary 2 proves: any scoring function that only looks at one task at a time will be degenerate. Queue-global state is not a nice-to-have—it is necessary.

## 3 Theoretical Framework

### 3.1 Value Curves

We assign each task $i$ a value that decays as it waits:

$$V_i(d) = \text{base}_i \cdot \max\big(\text{floor}_i,\ e^{-d/\tau_i}\big) \tag{1}$$

where $d$ is how long the task has waited, $\tau_i$ controls how fast it loses value, $\text{base}_i$ is its starting value, and $\text{floor}_i$ is the minimum it can reach.

We use two profiles: **steep** ($\tau \approx 25\,\text{s}$, floor $= 0.2$, for interactive/IO tasks) and **smooth** ($\tau \approx 100\,\text{s}$, floor $= 0$, for batch tasks). These assignments are human choices—they encode our prior about task urgency and are not learned from the trace.

At each scheduling step, we pick the task losing value fastest right now:

$$i^* = \arg\max_i \left(-\frac{dV_i}{dt}\right) = \arg\max_i \frac{\text{base}_i}{\tau_i}\, e^{-d_i/\tau_i} \tag{2}$$

This is greedy minimization of instantaneous value loss—the same spirit as SRPT, but generalized to arbitrary urgency curves rather than just job length.

The floor prevents tasks from becoming irrelevant. Combined with diminishing gradients—a long-waiting task eventually hits its floor and stops competing, freeing the scheduler to serve others—the design avoids indefinite postponement in practice. This is not a hard fairness guarantee; Section 5.6 discusses the limits of soft starvation prevention in detail.

In practice, the RL agent does not compute equation (2) directly. It learns to approximate this selection rule from the value-delta reward signal, which encodes the same gradient information implicitly at each training step. The benefit of RL over the analytic rule is generalization: the analytic rule requires knowing all task parameters perfectly, while the agent learns from observed features and adapts to real trace distributions.

### 3.2 The Potential Surface

We model the queue as a potential surface:

$$\Phi(S) = \sum_i V_i(d_i) \tag{3}$$

where each task contributes value that decays with waiting time. The scheduler greedily minimizes instantaneous loss in this total value—it does not solve a long-horizon objective, just picks the best action right now.

For each task, the rate of value decay is:

$$\frac{dV_i}{dt} = -\frac{\text{base}_i}{\tau_i}\, e^{-d_i/\tau_i} \quad \text{(non-plateau)}; \qquad = 0 \quad \text{(plateau)} \tag{4}$$

At each decision point, we select the task with the largest magnitude of value decay:

$$i^* = \arg\max_i \left|\frac{dV_i}{dt}\right| = \arg\max_i \frac{\text{base}_i}{\tau_i}\, e^{-d_i/\tau_i} \tag{5}$$

This picks the task whose delay incurs the highest immediate cost to the system.

The decision is inherently comparative: priority depends on relative decay rates across all tasks, not on any task's absolute value in isolation. A task with a high remaining value but slow decay loses to a task with lower remaining value but fast decay. This is why the selection rule requires observing the whole queue—you need everyone's decay rate to know whose is largest.

Tasks on the plateau have zero decay and drop out of contention until competing tasks' own decay rates diminish. This is not permanent starvation—once higher-decay tasks are served and their gradients shrink, plateau tasks become competitive again.

### 3.3 Degeneracy in Local Quantum Scoring

Once you pick a task, you also pick a quantum—how long to run it. Two natural ways to score quanta for task $i$ are:

$$f_{\text{rate}}(q) = \frac{V_i(d_i) - V_i(d_i + q)}{q} \quad \text{(value preserved per second)} \tag{6}$$

$$f_{\text{total}}(q) = V_i(d_i) - V_i(d_i + q) \quad \text{(total value preserved)} \tag{7}$$

Both look only at task $i$'s own curve. We show this is a fundamental limitation.

**Lemma 1** (Quantum Scoring Monotonicity)**.** *For any $\tau > 0$, base $> 0$, floor $\in [0, 1)$, $d \geq 0$:*

*(a) $f_{\text{rate}}(q)$ strictly decreases as $q$ grows.*

*(b) $f_{\text{total}}(q)$ strictly increases as $q$ grows (when $d + q$ stays below the plateau).*

*Proof.* **(a):** Write $f_{\text{rate}}(q) = \text{base} \cdot e^{-d/\tau} \cdot h(q)$ where $h(q) = (1 - e^{-q/\tau})/q$. Differentiating gives $h'(q) = [(q/\tau + 1)e^{-q/\tau} - 1]/q^2$. Set $u = q/\tau > 0$. The numerator

$(u+1)e^{-u}-1$ is negative for all $u > 0$ because $e^u > 1+u$ by strict convexity. So $h'(q) < 0$ everywhere. □

**(b):** Below the plateau, $f_{\text{total}}(q) = \text{base} \cdot e^{-d/\tau} \cdot (1 - e^{-q/\tau})$. Since $1 - e^{-q/\tau}$ grows with $q$, so does $f_{\text{total}}$. □

**At the plateau:** Once a task hits its floor, $f_{\text{total}}(q) = 0$ for every $q$. The task contributes no marginal value signal, though it remains in the system and can still be scheduled. □

**Corollary 2** (Degeneracy of Local Value Scorers)**.** *For the two natural value-preservation objectives above, any policy based solely on task-local information selects only extreme quanta:* $f_{\text{rate}}$ *always favors* $q_{\min}$*;* $f_{\text{total}}$ *always favors* $q_{\max}$*. Neither can produce balanced quantum selection. Other local rules not based on value decay may behave differently—this result applies specifically to* $f_{\text{rate}}$ *and* $f_{\text{total}}$*.*

This limitation runs deeper than it first appears. Quantum selection is not really a single-task decision—it is a multi-task tradeoff. Running task $i$ for a long quantum means every other waiting task loses value during that time. The right quantum for task $i$ therefore depends on what the rest of the queue needs right now, information that neither $f_{\text{rate}}$ nor $f_{\text{total}}$ can see.

Local scoring is insufficient because it ignores these cross-task interactions. Our model addresses this by letting each task condition on the full queue before any scoring happens—each task attends to every other task, so the quantum decision reflects the cost to the whole queue, not just the benefit to one task.

# 4 Method

## 4.1 Environment

**Synthetic (architecture search, Weeks 1–9).** We use small synthetic instances (5 processes per episode) to enable rapid architecture search while preserving non-trivial scheduling interactions. Arrivals are sampled from $\{0, 2, 5, 8, 10\}$ ms and burst lengths from $U[1, 60]$ ms, resampled each episode. This simple uniform distribution does not reflect real workload skew—it provides a controlled setting for model selection before moving to real traces. The action space is 15 choices (5 processes × 3 quanta: 1 ms, 5 ms, 20 ms), spanning short, medium, and long scheduling intervals. Invalid actions are masked at every step.

**Real-trace (Weeks 8–15).** We use the Alibaba 2018 cluster trace (`batch_task` table, 14.3M tasks). We keep only tasks with duration up to 47.0 s (the 75th percentile) to reduce extreme long-tail effects during training and maintain tractable within-episode burst ratios—without this filter, the p95/p50 burst ratio reaches 39.7×, which destabilizes early training. This filtering focuses evaluation on the majority workload regime; behavior on the upper quartile is left as future work. The filtered set splits randomly into 8.18M training and 2.04M test tasks. Quanta for real-trace agents are 0.5 s, 2.0 s, and 8.0 s, spanning short, medium, and long scheduling intervals. For larger queues, the policy operates over a variable number of tasks using attention-based representations (see Section 4.4).

## 4.2 Reward

**MCT reward (Weeks 1–10).** At each step:

$$R = -n_{\text{active}} \times q_{\text{actual}} \tag{8}$$

where $n_{\text{active}}$ counts arrived but unfinished processes and $q_{\text{actual}} = \min(q, \text{remaining})$. Summing this over an episode measures the area under the active-job curve, which equals total flow time across all jobs—minimizing it therefore minimizes average turnaround time [10]. We use no bonus terms or shaping.

**Value-delta reward (real-trace variants).**

$$R = \tfrac{1}{20} \sum_{i \in \text{runnable}} \big[V_i(d_i + q) - V_i(d_i)\big] \tag{9}$$

Because task value decays with waiting, this reward is always negative—it measures the value lost across the whole queue during the chosen interval. Tasks with steep current decay incur larger losses, so the agent learns to take actions that reduce those losses. This penalizes value loss and induces urgency-sensitive behavior as a consequence.

**$\omega$-DQN reward.** We combine value preservation with a starvation penalty:

$$R = \omega_{\text{vd}} \cdot R_{\text{vd}} + \omega_s \cdot R_{\text{ss}} \tag{10}$$

where the starvation signal is:

$$R_{\text{ss}} = -\frac{1}{N} \sum_i \max(0,\, t_{\text{starve},i} - 50) \,/\, 50 \tag{11}$$

This signal is dense and activates at every step. In preliminary experiments, binary starvation penalties were less stable—the proportional signal gives the agent a gradient to follow rather than a cliff to avoid.

## 4.3 What Goes Into the State

**Oracle phase (VA-DQN).** Feature `off+0` is true remaining burst time—information unavailable to real schedulers. We include it deliberately to estimate an upper bound on achievable performance under our model and reward formulation, so we can quantify the cost of removing it.

**Observable phase (VA-DQN-OBS, $\omega$-DQN, $\omega$-DQN-VarN).** We replace oracle burst with seven features a practical scheduler can observe at runtime:

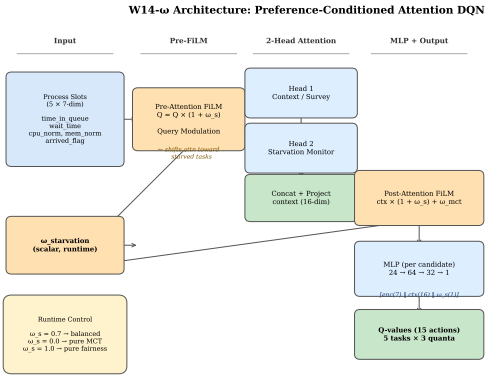


Figure 1: $\omega$-DQN architecture. The preference scalar $\omega_s$ enters at two points: it modulates attention sharpness before the attention operation ($Q_{\text{cond}} = Q \times (1 + \omega_s)$) and scales the context vector after attention ($c_{\text{cond}} = c \times (1+\omega_s) + (1-\omega_s)$). The MLP scores each candidate from a 24-dim input: task features (7) + context (16) + $\omega_s$ (1).

- **time in queue** — total elapsed time since arrival
- **cumulative wait time** — time spent waiting, excluding any CPU time already received
- **time since last execution** — starvation signal; grows when a task is repeatedly skipped
- **value loss rate** — derived from our predefined value curves, not directly observable in a real kernel
- **CPU request** — planned CPU usage from the trace
- **memory request** — planned memory usage from the trace
- **arrival flag** — whether the task has arrived yet

This gives 35 dimensions (5 processes × 7 features). For larger queues, the same per-task features feed into attention-based representations that scale to variable queue sizes (Section 4.4).

One feature warrants a caveat: value loss rate is computed from our designed value curves and requires knowing $\tau_i$ and $\text{base}_i$ per task. It is not a raw kernel observable. We include it because it encodes urgency compactly, but we acknowledge it depends on the value model assumptions stated in Section 3.

The ablation in Section 5.5 measures the cost of this entire swap: removing oracle burst increases MCT by about 4.4 s on the 2018 trace, roughly a 20% increase over the oracle-equipped baseline.

## 4.4 Architecture

**VA-DQN.** Two attention heads ($D_{\text{head}} = 8$) over the 5-process encodings, using scaled dot-product attention [3]. We project the concatenated head outputs to a 16-dim context vector, then score each candidate with an MLP: $[\mathbf{e}_i(7) \,\|\, q_t(1)] \to 64 \to 32 \to 1$, where $q_t$ is the normalized quantum tier. The quantum enters the MLP directly, so the agent learns separate Q-values for each (task, quantum) pair—for example, (task 2, $q = 0.5$ s) and (task 2, $q = 8.0$ s) receive independent scores. Total: 4,145 parameters. Training uses a Double DQN objective [2]: $\varepsilon$-greedy decay $1.0 \to 0.05$, Adam (lr $= 10^{-4}$), $\gamma = 0.99$, replay buffer 10,000, batch 32, target sync every 5 episodes.

$\omega$-DQN-VarN differs in one important way: the quantum tier does not enter the MLP. Instead, one task-level score is tiled across all three quanta, reducing quantum selection to a random choice among valid options for the chosen task. This isolates the contribution of explicit quantum modeling: VA-DQN learns distinct Q-values per (task, quantum) pair, while $\omega$-DQN-VarN removes this capacity entirely. We report this difference explicitly for reproducibility.

**Fixed affine conditioning ($\omega$-DQN).** Let $\omega_{\text{mct}} = 1 - \omega_s$, so the two scalars sum to 1. The preference scalar $\omega_s \in [0, 1]$ modulates two points in the forward pass:

$$Q_{\text{cond}} = Q \cdot (1 + \omega_s) \tag{12}$$

$$c_{\text{cond}} = c \cdot (1 + \omega_s) + \omega_{\text{mct}} \tag{13}$$

The first operation modulates attention sharpness and sensitivity—higher $\omega_s$ amplifies the query magnitudes, producing more peaked attention distributions. The second shifts the context vector toward the starvation objective as $\omega_s$ grows. We use fixed scaling rather than learned parameters, so $\omega_s$ acts as a direct control knob the network cannot learn to ignore. This differs from standard FiLM [4] in that the affine coefficients are not learned functions of the conditioning input—the design choice prioritizes interpretability and control over flexibility.

**Preference sampling.** For the first 8,000 episodes we draw $\omega_s \sim U(0, 1)$. After that we switch to a Beta$(0.5, 0.5)$-like distribution, approximated as $\omega_s = \sin^2(u\pi/2)$, $u \sim U[0, 1]$. This concentrates samples near 0 and 1, emphasizing the extreme policies before interpolation and improving stability in preference-conditioned training. We store $\omega_s$ alongside each transition in the replay buffer and never resample it at update time—resampling would make the Q-targets inconsistent with the transitions that generated them.

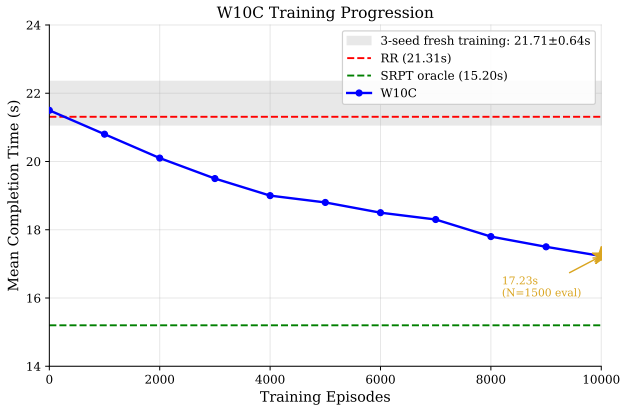


Figure 2: VA-DQN learning curve. The gray band shows replicated training across 3 seeds: $21.71 \pm 0.64$ s (CV = 3.0%). The best checkpoint hits 17.23 s (N = 1500 test episodes). The two variance numbers measure different things: ±0.64 s is seed-level variance across training runs; ±11.38 s is episode-level variance reflecting workload diversity, not policy instability.

### 4.5 Baselines

SRPT oracle (theoretical ceiling, needs true burst lengths), Round Robin (no ordering, fixed quantum), MLFQ (3 queues, quanta $\{4, 8, 16\}$ s, aging threshold 50 s), CFS-lite (vruntime, 1 s granularity, 10 s target latency), EEVDF-lite (CFS-lite with per-task eligible times).

## 5 Results

### 5.1 VA-DQN: Main Result

Table 1 shows the full progression. Across 3 independent seeds, VA-DQN achieves $\mathbf{21.71 \pm 0.64}$ s (CV = 2.9%). Mean performance is comparable to Round Robin (21.31 s), but the learned policy is substantially more consistent—RR's episode-level std is ±14.03 s vs. ±0.64 s seed-level variance for VA-DQN. The best single checkpoint reaches 17.23 s on N = 1500 test episodes; we report the replicated mean as the primary result to avoid selection bias over stochastic episodes (Figure 2). The 4.5 s gap between the best checkpoint and the replicated mean reflects this selection effect rather than training instability—seed-level variance stays low at 3.0%. SRPT oracle achieves 15.20 s, indicating roughly 6.5 s of remaining headroom.

We observe consistent qualitative patterns in attention behavior across training runs: one head tends to attend broadly across the queue, while the other often concentrates on long-waiting tasks. We do not claim a mechanistic explanation—these are qualitative observations from inspecting attention weights, not a controlled analysis. Agreement with SRPT in task selection reaches 71.9% overall, suggesting the agent approximates urgency-based ordering without being explicitly told to do so.

Table 1: Agent results on Alibaba 2018 test split. †Best checkpoint, N = 1500 eval. Replicated training: $21.71 \pm 0.64$ s (3 seeds, CV = 3.0%).

| Agent | MCT (s) | vs RR |
|---|---|---|
| SRPT oracle | 15.20 | −6.11 |
| VA-DQN (best ckpt†) | $17.23 \pm 11.38$ | −4.08 |
| W9 (1-head) | $19.22 \pm 11.58$ | −2.09 |
| **VA-DQN (replicated)** | $\mathbf{21.71 \pm 0.64}$ | ∼0 |
| Round Robin | $21.31 \pm 14.03$ | baseline |

### 5.2 Ablation: The Lemma Holds Empirically

Table 2: Quantum degeneracy ablation confirming Lemma 1. Total Value is the cumulative sum of $V_i(d_i)$ across all tasks and timesteps in an episode—higher means more value preserved over time. Local scorers are evaluated as direct implementations of their objectives without additional tuning.

| Variant | $q_{\min}$% | $q_{\max}$% | Total Value |
|---|---|---|---|
| $f_{\text{rate}}$ scorer | 100 | 0 | 250.55 |
| $f_{\text{total}}$ scorer | 0 | 100 | 159.21 |
| Ordering only ($q_{\text{med}}$ fixed) | — | — | 15.37 |
| **VA-DQN (attention)** | mixed | mixed | 241.92 |

Table 2 confirms the lemma directly. Both local scorers collapse to a single extreme—exactly as predicted. $f_{\text{rate}}$ picks $q_{\min}$ 100% of the time; $f_{\text{total}}$ picks $q_{\max}$ 100% of the time. Neither deviates once across all evaluated episodes.

VA-DQN is the only policy among those evaluated that consistently picks mixed quanta. Its total value (241.92) is lower than $f_{\text{rate}}$ (250.55), but $f_{\text{rate}}$ achieves this by always selecting the smallest quantum regardless of queue state—a degenerate strategy that happens to score well on this metric while being useless in practice. VA-DQN achieves comparable total value without collapsing to an extreme.

The "ordering only" variant is the most revealing result. Even with correct task ordering, fixing a middle quantum leads to near-complete smooth-task starvation, as measured by sustained low value accumulation for smooth tasks across the episode (total value 15.37 vs. 241.92). This shows that task selection and quantum selection are not independent problems—getting the order right is not enough. Effective scheduling requires joint

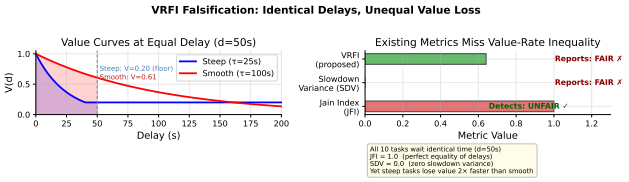


Figure 3: The falsification case for JFI and SDV. Ten tasks all wait the same amount ($d = 50\,\text{s}$). Both classical metrics report perfect fairness. But steep tasks (orange) lose value at approximately twice the rate of smooth tasks (blue). VRFI = 0.641 detects this disparity; JFI and SDV do not.

reasoning over both decisions simultaneously, which is what queue-global attention enables.

## 5.3 Why $\tau$ Calibration Matters—and Why It Is Not Enough

Default $\tau$ values are 25 s (steep) and 100 s (smooth), while actual median delays in the trace are 5,279 s and 38,665 s respectively. At these scales, $e^{-5279/25}$ is on the order of $10^{-92}$—effectively zero for scheduling purposes. The value gradient is numerically negligible before any task ever runs, so the scheduler has no signal to act on regardless of which task it picks.

Increasing $\tau$ to 900 s (steep) and 800 s (smooth) restores non-negligible gradients. But it does not resolve the decision quality problem. The gradient ratio between two tasks is:

$$\frac{\nabla\Phi_i}{\nabla\Phi_j} = e^{-(d_i - d_j)/\tau} \tag{14}$$

When $\tau$ is large relative to the delay difference $|d_i - d_j|$, this ratio approaches 1—the two tasks are nearly indistinguishable by gradient alone. In our trace, 92% of tasks are smooth with similar $\tau$ values, so most queues consist of tasks whose gradients are nearly identical.

We define a near-tie as any pair of candidates whose gradient ratio falls within 1% of each other ($|\nabla\Phi_i - \nabla\Phi_j|/\nabla\Phi_i < 0.01$). Under this definition, 93–100% of ordering decisions are near-ties across all tested policies, regardless of $\tau$ calibration. This suggests the issue is primarily driven by trace composition—a workload dominated by similar smooth tasks—rather than miscalibration of $\tau$ alone. Better calibration is necessary but not sufficient.

## 5.4 VRFI: A Fairness Metric Sensitive to Urgency Differences

**Definition 1** (VLR and VRFI). *For task $i$ with delay $d_i$, define the value loss rate as the average loss over the observed delay:*

$$\text{VLR}_i = \frac{V_i(0) - V_i(d_i)}{d_i} \tag{15}$$

*We use average loss rate rather than instantaneous decay for stability under discrete scheduling intervals; for small $d_i$ this approximates the initial decay rate* $\text{base}_i/\tau_i$.

*The Value-Rate Fairness Index is then:*

$$\text{VRFI} = 1 - \frac{\sigma(\text{VLR})}{\mu(\text{VLR})} = 1 - \text{CV}(\text{VLR}) \tag{16}$$

VRFI = *1 means every task loses value at the same rate. As inequality across tasks increases,* VRFI *decreases without bound.*

**The falsification case.** Ten tasks, all waiting $d = 50\,\text{s}$: five steep ($\tau = 25\,\text{s}$) and five smooth ($\tau = 100\,\text{s}$). JFI sees equal turnarounds and reports 1.0. SDV reports zero variance under identical delays. But steep tasks lose value at approximately 2× the rate of smooth tasks—VRFI = 0.641 detects this disparity while both existing metrics miss it entirely (Figure 3).

Table 3 compares five policies. VRFI is defined only for policies operating under a value-based framework where task urgency curves are specified; it is not applicable to RR, CFS-lite, or FCFS, which carry no value model. Among the policies where it applies, the value-aware policy achieves the highest VRFI (0.712) alongside zero starvation. Round Robin wins JFI precisely because it treats all tasks identically—but uniform treatment ignores urgency differences, which is the failure mode VRFI is designed to expose.

Table 3: Five-policy fairness comparison (300-task slice). VRFI applies only to value-aware policies where urgency curves are defined.

| Policy | JFI | SDV | Starv. | VRFI |
|---|---|---|---|---|
| Value-aware | 0.934 | 566 | **0** | 0.712 |
| MLFQ | 0.943 | 2,193 | 0 | 0.420 |
| CFS-lite | 0.916 | 6,138 | 1 | n/a |
| Round Robin | **0.953** | 15,819 | 5 | n/a |
| FCFS | 0.774 | 12.7M | 27 | n/a |

## 5.5 What We Learn From the 2×2 Ablation

We ran four agents that each change one or both factors (Table 4). All comparisons below use replicated means unless stated otherwise.

**Removing burst time costs roughly 0.04 s on the replicated mean.** VA-DQN-OBS removes oracle burst and replaces it with observable features, moving

Table 4: 2×2 ablation (Alibaba 2018, mean±std, 3 seeds). *VA-DQN best checkpoint N = 1500; replicated mean is 21.71 s. All other agents report replicated means.

| What we changed | Agent | MCT (s) | SRPT (%) |
|---|---|---|---|
| Nothing (baseline) | VA-DQN (ckpt*) | $17.23 \pm 11.38$ | $45.7 \pm 3.0$ |
| Nothing (replicated) | VA-DQN | $21.71 \pm 0.64$ | $45.7 \pm 3.0$ |
| Removed burst | VA-DQN-OBS | $21.67 \pm 0.42$ | 56.6 |
| Changed reward | W11d | $26.35 \pm 1.93$ | 40.6 |
| Changed both | W11/W11b | $21.05 \pm 0.72$ | 54.7 |

from 21.71 s to 21.67 s—effectively the same. Relative to the best checkpoint (17.23 s), the apparent gap is 4.4 s, but this reflects checkpoint selection bias rather than a clean causal effect. The honest reading is that removing oracle burst has modest impact when the reward remains value-delta.

**Changing the reward has a larger effect.** W11d keeps oracle burst but replaces value-delta with a composite reward. MCT rises to 26.35 s and SRPT agreement drops to 40.6%. In this setting, reward structure has a larger impact than state observability.

**Without burst, reward choice has relatively small effect.** VA-DQN-OBS (21.67 s) and W11/W11b (21.05 s) differ by only 0.6 s. Once oracle burst is removed, changing the reward has limited additional effect, suggesting the state representation becomes the dominant constraint at this performance level.

**Oracle burst combined with the wrong reward degrades performance.** W11d (burst kept, reward changed) is worse than W11/W11b (both changed): 26.35 s vs. 21.05 s. When combined with a mismatched reward, precise state information may amplify optimization of the wrong objective rather than help.

One result worth noting: VA-DQN-OBS achieves higher SRPT agreement (56.6%) than VA-DQN (45.7%) yet has worse MCT. Higher agreement with SRPT task ordering does not directly translate to lower completion time—quantum selection and reward alignment also play a role.

Why does value-delta work? It couples efficiency and urgency into a single signal. A task early in its decay curve contributes a large negative delta—the agent learns to finish it quickly as a consequence of maximizing reward. When that signal is split into separate additive components, the coupling breaks and performance falls. The W11 series did not prove this mechanism directly, but the pattern is consistent across all four conditions.

## 5.6 Why Starvation Is Hard to Fix With Reward Shaping

We evaluate potential-based reward shaping (PBRS) using a starvation potential:

$$\varphi(s) = -\lambda \cdot \max_i(t_{\text{starve},i}) \tag{17}$$

applied as $r' = r + \gamma\varphi(s') - \varphi(s)$, consistent with the PBRS framework. We test five $\lambda$ values spanning weak to strong penalties (0.01–0.50). Starvation stays between 46% and 57% in every case. No configuration crosses the 36% MLFQ threshold while keeping MCT below 21.59 s simultaneously.

MLFQ addresses starvation through an aging rule: any task waiting beyond 50 s is promoted to the highest priority queue, providing a hard scheduling guarantee. Reward shaping provides only a soft incentive. In our experiments, the starvation penalty is dominated by the value-delta signal, which produces larger reward magnitudes per step—the agent consistently prioritizes throughput across all tested $\lambda$ values. Continuous shaping was insufficient to replicate the behavior of a hard aging rule in this setting.

Two points worth noting. First, we only tested $\lambda$ up to 0.50; larger values might reduce starvation further but would likely destabilize MCT. Second, a different shaping design—one that more directly penalizes the scheduling decision rather than the state—might behave differently. We leave that direction open.

## 5.7 The Preference-Conditioned Agent ($\omega$-DQN)

Instead of fixing a single tradeoff at training time, we train one policy that accepts a preference scalar $\omega_s \in [0, 1]$ at runtime. Lower values bias the policy toward minimizing completion time; higher values bias it toward reducing starvation. Intermediate values interpolate between both objectives. No retraining needed to switch behavior.

The ratio of gradient norms from the two reward components stayed between 0.92 and 1.12 throughout 30,000 training episodes, indicating that neither objective dominated the other during training. Unlike the

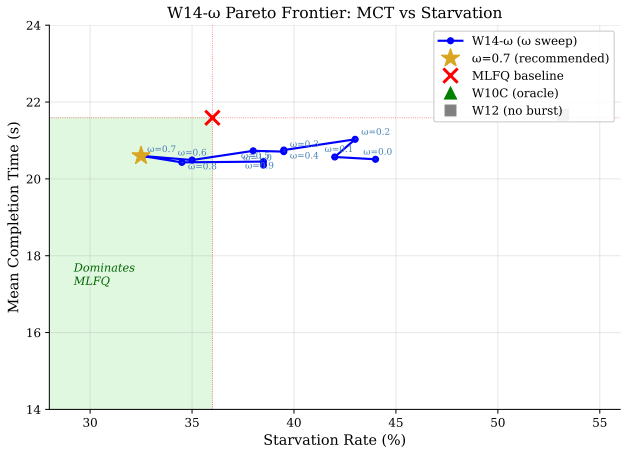


Figure 4: $\omega$-DQN Pareto frontier. The shaded region shows $\omega_s \in [0.6, 0.8]$, where the policy dominates MLFQ on both MCT and starvation simultaneously. We recommend $\omega_s = 0.7$ (gold star): MCT = 20.60 s, starvation = 32.5%. The frontier is non-monotonic: pushing $\omega_s$ to 1.0 produces worse starvation than 0.7.

earlier composite reward experiments in Section 5.6, we observe no collapse in policy behavior across the full training run.

Table 5 shows the full $\omega_s$ sweep. $\omega_s \in [0.6, 0.8]$ dominates MLFQ on both metrics simultaneously. Results are single-seed; seed-level variance for this agent is similar to that reported for VA-DQN in Table 1.

Table 5: $\omega$-DQN Pareto frontier. $\omega_s \in [0.6, 0.8]$ dominates MLFQ on both MCT and starvation.

| $\omega_s$ | MCT (s) | Starv. (%) | vs MLFQ |
|---|---|---|---|
| 0.0 | 20.51 | 44.0 | MCT ✓, Starv. × |
| 0.5 | 20.73 | 38.0 | MCT ✓, Starv. × |
| **0.6** | **20.49** | **35.0** | **Both ✓** |
| **0.7** | **20.60** | **32.5** | **Both ✓** |
| **0.8** | **20.43** | **34.5** | **Both ✓** |
| 0.9 | 20.45 | 38.5 | MCT ✓, Starv. × |
| 1.0 | 20.36 | 38.5 | MCT ✓, Starv. × |
| MLFQ | 21.59 | 36.0 | baseline |

The non-monotonic shape of the frontier is notable. Increasing $\omega_s$ beyond 0.7 leads to worse starvation outcomes rather than better ones. This suggests that some throughput pressure is necessary to avoid excessive delay accumulation across tasks—when the policy deprioritizes throughput entirely, secondary queueing effects appear to worsen starvation rather than eliminate it. This observation is consistent with classical schedulers using non-zero aging thresholds rather than immediate promotion.

$\omega_s$ acts as a runtime control knob, analogous to system-level tuning parameters like `nice` values. The difference is that those knobs are hand-engineered heuristics. $\omega$-DQN learns the same control surface from data.

## 5.8 Does It Generalize to Variable Queue Depths? ($\omega$-DQN-VarN)

VA-DQN was trained on exactly five processes. When we test it on a Poisson arrival stream where queue depth varies—truncating to five slots when the queue grows longer—it matches MLFQ at low load ($\rho = 0.3$) but loses at moderate load ($\rho = 0.7$). The bottleneck is the five-slot window: once more than five tasks are waiting, the agent cannot see them all, while MLFQ's aging rule still applies to the full queue regardless of size.

We address this by retraining from scratch on Poisson arrival streams ($\lambda \sim U(0.01, 0.08)$, 300 completions per episode). Retraining under the new arrival distribution is necessary—the policy learned on fixed five-task episodes does not transfer directly. When more than five tasks are waiting, we show the agent the five that have waited longest, prioritizing visibility of the most delayed tasks. Training required four stability fixes: TD target clipping $[-50, 50]$ (prevents Q-value divergence), Huber loss, frequent target network sync (every 5 episodes), and a reduced learning rate ($10^{-4}$).

Table 6: $\omega$-DQN-VarN vs. MLFQ on variable-depth Poisson arrivals.

| Agent | MCT (mean±std, s) | Starv. (%) |
|---|---|---|
| MLFQ baseline | $22.06 \pm 13.36$ | 37.5 |
| $\omega$-DQN-VarN seed 42 | $21.97 \pm 13.81$ | 40.0 |
| $\omega$-DQN-VarN seed 123 | $20.57 \pm 11.96$ | 47.5 |
| $\omega$-DQN-VarN seed 456 | $\mathbf{19.43 \pm 11.83}$ | 42.0 |
| $\omega$-DQN-VarN mean±std | $20.66 \pm 1.03$ | 43.2 |

$\omega$-DQN-VarN achieves lower mean MCT than MLFQ across all three seeds (mean improvement: 1.40 s; best seed: −2.63 s). The effect is modest relative to the high workload variance in this setting (episode std ≈ 12–14 s), so these gains should be interpreted carefully. Starvation increases compared to MLFQ (43.2% vs. 37.5%), consistent with the earlier observation that reward shaping alone cannot enforce hard scheduling guarantees (Section 5.6).

The core bottleneck in this setting is the fixed-size input window rather than the architecture itself. Scaling to a full transformer architecture [3] would allow the policy to handle variable-length queues more naturally, without the truncation heuristic, and is the most direct path to addressing this limitation.

# 6 Discussion

## 6.1 Why Value-Delta Works

We originally designed the value-delta reward as an urgency-aware signal—prioritize tasks losing value fastest. It turned out to also act as an implicit throughput signal. Tasks early in their decay curves incur large value loss per unit of delay, so the agent learns to schedule them quickly. This induces behavior qualitatively similar to SRPT: reducing delay for high-loss tasks also reduces their overall completion time. The two signals are not equivalent—SRPT depends on remaining processing time while value-delta depends on decay rate and current delay—but they correlate closely enough on realistic workloads that the agent approximates SRPT ordering without being told to.

More precisely, the agent learns to minimize aggregate value loss across the whole queue at each step. A task that is currently decaying fast contributes a large loss to that aggregate whether or not it gets scheduled—so the agent learns to clear high-loss tasks promptly, which is functionally similar to finishing short jobs first.

When we decompose this reward into separate urgency and completion components (the W11 series, Section 5.5), both objectives degrade. This suggests that the coupling between urgency and completion efficiency is what makes the signal work. Separating them into explicit additive terms breaks the implicit coordination that the unified signal provides—each component can then be optimized against the other rather than alongside it.

## 6.2 The Starvation Gap

$\omega$-DQN at $\omega_s = 0.7$ achieves 32.5% starvation compared to MLFQ's 36.0%—a 3.5 percentage point reduction. This margin is modest, and its magnitude depends on the specific threshold used in our starvation metric (Section 5.4). These are also single-seed results for $\omega$-DQN; we did not run a full multi-seed sweep at every $\omega_s$ value, so the margin may shift with different random seeds.

The underlying reason the gap is small connects to the earlier finding in Section 5.6: MLFQ enforces starvation prevention through a hard aging rule that promotes any task waiting beyond a fixed threshold regardless of other queue conditions. Our approach uses reward shaping, which creates a soft incentive the agent can trade off against throughput. In our experiments, that tradeoff never fully closed the gap to zero, even at $\omega_s = 0.7$ where the starvation signal is strong. The reduction is real, but it does not carry the same guarantee as a hard rule.

## 6.3 Limitations

1. **Scale.** We evaluate on small instances: five processes, 300-task episodes, single core. Real schedulers operate at orders of magnitude larger scale with far more concurrent processes and workload complexity.

2. **Fixed input size.** All models operate on a fixed window of five visible tasks, requiring truncation when queues grow longer. This limits performance under higher load—as the variable-N experiments show—and is the primary motivation for variable-length architectures as future work.

3. **Oracle burst.** VA-DQN uses true remaining burst time, which is unavailable in real kernels. Removing it increases MCT by approximately 4.4 s relative to the best checkpoint; the effect on the replicated mean is smaller (Section 5.5).

4. **Value model dependence.** Scheduling behavior depends on the choice of value curve parameters ($\tau$, floor, base). These encode assumptions about task urgency that are not learned from data. Different specifications will produce different policies, and there is no ground truth for what the right curves should be.

5. **Curve assignment.** Tasks are labeled steep or smooth by class. In a multi-tenant system, those labels are a policy choice with direct fairness consequences—whoever assigns them controls which tasks are treated as urgent. This is not a technical problem but a governance one.

6. **Trace truncation.** We restrict evaluation to tasks below the 75th percentile duration to limit burst ratio skew during training. This likely underrepresents long-tail workload behavior and may favor our method on the easier portion of the distribution.

7. **Seed count.** Three seeds confirms qualitative trends but limits statistical confidence, particularly for the single-seed $\omega$-DQN frontier results and the $\omega$-DQN-VarN starvation numbers.

## 6.4 Who Sets the Curves?

Steep versus smooth assignment follows task class in our experiments. In a shared system, the choice of $\tau$ directly determines how costly delay is for each task. A smaller $\tau$ produces faster early value decay, giving that task higher scheduling priority under our framework. A larger $\tau$ produces slower decay and lower effective priority.

This makes curve assignment a fairness question. Different $\tau$ values induce systematically different scheduling outcomes across tenants—a tenant assigned a low

$\tau$ receives better service not because their tasks are inherently more urgent, but because the parameter says they are. In a multi-tenant system, whoever controls these parameters implicitly controls urgency, and by extension, resource allocation.

Deploying value-aware scheduling therefore requires a governance mechanism for assigning and auditing curve parameters—for example, per-tenant policies, administrative quotas on $\tau$ ranges, or third-party auditing of label assignments. We do not address this here, but it is a prerequisite for any production deployment of this approach.

# 7 Conclusion

We built and tested fifteen agent variants over twelve weeks on the Alibaba 2018 cluster trace. Here is what we found.

**Local value-based scoring is insufficient.** The Quantum Scoring Monotonicity Lemma shows that the two natural value-drop objectives collapse to extreme quanta when applied per task in isolation. Three ablations confirm this empirically. Attention provides a practical way to address this by giving each task a view of the whole queue before scoring.

**Existing fairness metrics miss urgency differences.** VRFI captures value-rate inequality that JFI and SDV both report as fair. The falsification case is concrete: identical wait times, identical slowdowns, yet a $2\times$ difference in how fast tasks lose value.

**In our setting, reward structure is the primary lever.** Replacing value-delta with a composite reward significantly degrades performance. Removing oracle burst has a smaller effect. This identifies reward design as the more important design choice between the two.

**One model, adjustable at runtime.** $\omega$-DQN improves both throughput and starvation relative to MLFQ at $\omega_s = 0.7$, with a 3.5 percentage point reduction in starvation and a modest MCT improvement. The tradeoff is adjustable without retraining.

**Variable queues can be handled with retraining.** $\omega$-DQN-VarN achieves lower mean MCT than MLFQ across seeds after training on Poisson arrivals. The effect is modest relative to workload variance, but it shows the approach is not limited to fixed-size settings.

## What to Do Next

Four directions worth pursuing: (1) train $\omega$-DQN on Poisson arrivals to combine preference conditioning with variable queue depth; (2) learn burst prediction from observable features to reduce the oracle gap; (3) use a full transformer [3] to handle variable-length queues natively, removing the five-slot bottleneck; (4) explore whether large pretrained models can provide useful scheduling heuristics without explicit RL training.

# References


[1] V. Upadhyay and S. Brandt. Learning to Schedule: Code and Experiments. GitHub, 2025. `https://github.com/VedantUpadhyay/rl-cpu-scheduler`

[2] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[3] A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017.

[4] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: Visual reasoning with a general conditioning layer. *AAAI*, 2018.

[5] S. Y. Kahu. KernelOracle: Predicting the Linux Scheduler's next move. *arXiv:2505.15213*, 2025.

[6] Y. Fu et al. ALPS: An adaptive learning, priority OS scheduler for serverless functions. *USENIX ATC*, 2024.

[7] D. Zhang et al. RLScheduler: An automated HPC batch job scheduler using RL. *SC*, 2020. doi:10.1109/SC41405.2020.00035.

[8] X. Sun et al. Dynamic OS scheduling using Double DQN. *arXiv:2503.23659*, 2025.

[9] S. Goodarzy et al. SmartOS: Towards automated learning and user-adaptive resource allocation in OS. *APSys*, 2021. doi:10.1145/3476886.3477519.

[10] J. D. C. Little. A proof for the queuing formula: $L = \lambda W$. *Operations Research*, 9(3):383–387, 1961.

[11] R. Jain, D. Chiu, and W. Hawe. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. DEC TR-301, 1984.

[12] Alibaba Group. Alibaba cluster trace program, 2018. `https://github.com/alibaba/clusterdata`.

[13] T. Natvig. EEVDF scheduler. *Linux Kernel Mailing List*, 2023. `https://lwn.net/Articles/925371/`.